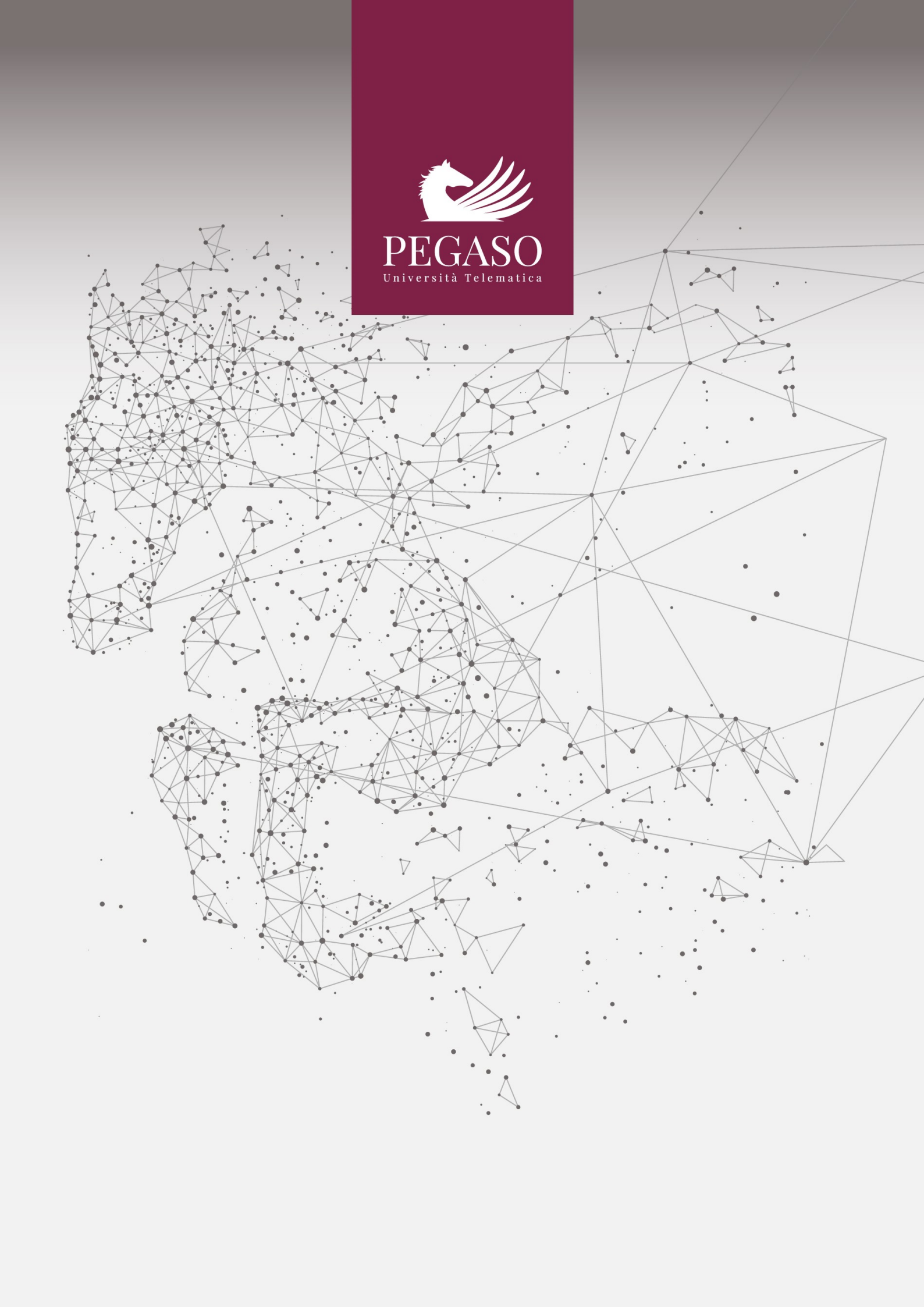

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. OPERAZIONI DI BASE SULLE STRINGHE 4

3. TEST. RICERCA MANIPOLAZIONE DI STRINGHE 8

4. ESERCIZI CON LE STRINGHE – CALCOLO DEL CODICE FISCALE 14

4.1 ANALISI DEL COGNOME – 3 CARATTERI ALFABETICI 14
4.2 ANALISI DEL NOME – 3 CARATTERI ALFABETICI 15
4.3 ANALISI DELLA DATA DI NASCITA E SESSO – 5 CARATTERI ALFANUMERICI 16
4.4 ANALISI DEL COMUNE (O STATO) DI NASCITA – 4 CARATTERI ALFANUMERICI 17
4.5 CARATTERE DI CONTROLLO – 1 CARATTERE ALFABETICO 18

BIBLIOGRAFIA 24

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si approfondiscono le stringhe insieme ai metodi, alle funzioni e alle potenzialità offerte da Python. In particolare, oltre alle operazioni di base sulle stringhe, si illustrano metodi e funzioni per la verifica di proprietà da parte di una stringa, per la ricerca di stringhe in altre stringhe e per la manipolazione di stringhe al fine di ottenere elaborazioni necessarie per risolvere particolari problemi. Viene, infatti, illustrato, in dettaglio, il problema e l'algoritmo di risoluzione del calcolo del codice fiscale che rappresenta uno dei casi più noti di manipolazione di una stringa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni di base sulle stringhe

La **stringa** è uno dei tipi di dato fondamentali e costitutivi di Python. Attraverso le stringhe, infatti, è possibile migliorare l'interazione tra programma e utente. Per le operazioni viste finora, l'uso delle stringhe è stato limitato all'input e all'output.

Ci sono diversi tipi di programma, però, che eseguono operazioni sulle stringhe (es. programmi di posta elettronica, motori di ricerca, ecc.).

Python mette a disposizione funzioni e metodi molto importanti per l'esame e la manipolazione delle stringhe. Molti concetti tipici delle sequenze di dati saranno validi anche per le stringhe.

In Python, le stringhe, di tipo **str**, sono delimitate (all'inizio e alla fine) da uno dei seguenti simboli:

1. Apice singolo ‘
2. Triplo apice singolo '''
3. Apice doppio "
4. Triplo apice doppio " " "

L'utilizzo di diversi simboli per la stessa funzionalità è stato reso disponibile per due scopi principali:

1. utilizzare caratteri di delimitazione delle stringhe all'interno del letterale di stringa (es. "c'era una volta");
2. utilizzare stringhe multilinea (es. """Lorem ipsum dolor sit amet elit """).

Per visualizzare a schermo le stringhe, si utilizza la funzione print(). Le stringhe sono vettori/sequenze di caratteri in codifica Unicode (16 bit per carattere per un totale di 65.536 possibili caratteri).

Ad ogni modo, in Python non esiste il tipo carattere; un carattere in Python, infatti, è semplicemente una stringa lunga uno.

A volte può essere utile accedere ai singoli caratteri di una stringa. Si pensi all'esame di una stringa per l'analisi dei vincoli su una password come lettera maiuscola, numero, lettera speciale, ecc. L'accesso ai singoli caratteri di una stringa può avvenire in due modi:

1. **ciclo for;**
2. **indicizzazione.**

Il **ciclo for** per le stringhe funziona esattamente come per le sequenze. Il codice seguente riporta un esempio di codice sorgente che accede a ciascun carattere della stringa.

```
stringa = "sole"
for a in stringa:
    print(a)
```

Il risultato dell'elaborazione del codice precedente risulta essere il seguente:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
s
o
l
e
```

Ovviamente, oltre alla stampa dei caratteri, il ciclo for può essere utilizzato per esaminare le stringhe. Un esempio potrebbe essere quello di contare le occorrenze di una determinata lettera.

Nel codice sorgente che segue si contano le r presenti in una parola.

```
stringa = "ramarro"
c=0
for a in stringa:
    if a=='r' or a=='R':
        c+=1
print("Nella stringa", stringa,"ci sono",c,"r")
```

Il risultato dell'elaborazione del codice precedente risulta essere il seguente:

```
Nella stringa ramarro ci sono 3 r
```

Secondo la tecnica dell'**indicizzazione**, invece, ogni carattere di una stringa ha un **indice** che ne specifica la posizione all'interno della stringa stessa. Gli indici partono, come per le sequenze, da 0 e arrivano fino alla lunghezza della stringa ridotta di uno. Il codice sorgente che segue mostra la stampa del primo (indice = 0), quarto (indice = 3) e sesto (indice = 5) carattere della stringa "ramarro".

```
stringa = "ramarro"
print(stringa[0],stringa[3],stringa[5])
```

Il risultato dell'elaborazione del codice precedente risulta essere il seguente:

```
r a r
```

Anche per le stringhe, come per le sequenze, sono validi gli indici negativi. L'indice di valore -1 corrisponderà all'ultimo carattere, l'indice di valore -2 al penultimo e così via. Il codice sorgente che segue mostra la stampa dell'ultimo, del terzultimo e del quintultimo carattere della stringa "ramarro".

```
stringa = "ramarro"
print(stringa[-1],stringa[-3],stringa[-5])
```

Il risultato dell'elaborazione del codice precedente risulta essere il seguente:

```
o r m
```

Se l'indice utilizzato, invece, è al di fuori dell'intervallo, verrà sollevata un'eccezione **IndexError** come mostrato nel codice seguente in cui si cerca di accedere all'undicesimo carattere (indice = 10) della stringa "ramarro" che contiene, però, solo 7 caratteri.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
stringa = "ramarro"
print(stringa[10])
```

L'elaborazione del codice precedente produce il seguente messaggio di traceback:

```
Traceback (most recent call last):
  File "C:\Users\claud\AppData\Local\Programs\Python\Python39\prove.py", line 2,
 in <module>
    print(stringa[10])
IndexError: string index out of range
```

Per evitare l'eccezione IndexError è necessario procedere secondo una delle seguenti due modalità:

1. verificare l'appartenenza all'intervallo di indici attraverso la funzione len() di stringa come nel seguente codice sorgente:

```
index = 10
stringa = "ramarro"
if index<len(stringa):
    print(stringa[10])
else:
    print("Indice non valido")
```

2. gestire l'eccezione IndexError come nel seguente codice sorgente:

```
index = 10
stringa = "ramarro"
try:
    print(stringa[10])
except:
    print("Indice non valido")
```

In entrambi i casi, in caso di indice fuori intervallo, verrà visualizzata a schermo la seguente stringa:

```
Indice non valido
```

Un'operazione molto importante con le stringhe è il **concatenamento** (**operatore +**). È possibile concatenare due o più stringhe (ottenute anche come casting a stringa). Nel codice sorgente che segue vengono dichiarate e concatenate tre stringhe (l'ultima ottenuta come casting a stringa del numero 20):

```
s1="Ciao "
s2="Claudio, sono le ore "
s3 = str(20)
s = s1+s2+s3
print(s)
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del codice precedente prevede la visualizzazione a schermo della seguente stringa:

```
Ciao Claudio, sono le ore 20
```

Analogamente alle sequenze, una porzione di stringa è costituita da un intervallo di caratteri appartenenti alla stringa stessa. Le porzioni di stringhe prendono il nome di **substringhe** o **sottostringhe**. Nel codice sorgente che segue, viene dichiarata una stringa *s* e, successivamente, vengono estratte delle sottostringhe:

1. s1 sottostringa di s dal carattere di indice 8 fino al termine della stringa;
2. s2 sottostringa uguale a s1 senza indicazione esplicita del termine della stringa;
3. s3 sottostringa di s dal primo carattere fino al carattere di indice 7;
4. s4 sottostringa di s che prende tutti i caratteri dispari della stringa s.

```
s = "Stasera che sera"
s1 = s[8:len(s)]
s2 = s[8:]
s3 = s[:7]
s4 = s[0:len(s):2]

print("s1 =",s1)
print("s2 =",s2)
print("s3 =",s3)
print("s4 =",s4)
```

Il risultato dell'elaborazione del codice sorgente precedente sarà:

```
s1 = che sera
s2 = che sera
s3 = Stasera
s4 = Saeacesr
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Test. Ricerca manipolazione di stringhe

Come per le sequenze, anche per le stringhe Python prevede l'utilizzo dell'**operatore in** e **not in**. Nel codice sorgente che segue, nella stringa "nomi" sono riportati 6 nomi di persona. Viene, quindi, richiesto all'utente di inserire un nome da ricercare. In base al risultato dell'operatore in, verrà visualizzato a schermo un messaggio di presenza o di assenza del nome cercato nell'elenco dei nomi.

```
nomi = "Antonio Bice Carlo Deborah Fabio Giorgia"
n = input("Inserire nome da ricercare: ")
if n in nomi:
    print(n,"è nell'elenco")
else:
    print(n,"non è nell'elenco")
```

Nel caso di esempio in cui viene inserito il nome "Carlo", il messaggio sarà di presenza del nome nella stringa.

```
Inserire nome da ricercare: Carlo
Carlo è nell'elenco
```

Nel caso di esempio in cui viene inserito il nome "Sergio", il messaggio sarà di assenza del nome nella stringa.

```
Inserire nome da ricercare: Sergio
Sergio non è nell'elenco
```

Di fondamentale importanza è ricordare che le stringhe sono oggetti con metodi che servono a:

1. **testare i valori delle stringhe;**
2. **eseguire manipolazioni;**
3. **cercare sottostringhe e sostituire sequenze di caratteri**.

I metodi per il **test delle stringhe**:

- iniziano generalmente con is;
- non richiedono parametri;
- restituiscono un valore booleano True o False.

I metodi per il test delle stringhe più importanti sono:

- **isalnum**() restituisce True se la stringa contiene solo caratteri alfanumerici e ha lunghezza >=1;
- **isalpha**() restituisce True se la stringa contiene solo caratteri alfabetici e ha lunghezza >=1;
- **isdigit**() restituisce True se la stringa contiene solo caratteri numerici e ha lunghezza >=1;
- **islower**() restituisce True se la stringa contiene tutti i caratteri alfabetici minuscoli e ha lunghezza >=1;
- **isupper**() restituisce True se la stringa contiene tutti i caratteri alfabetici maiuscoli e ha lunghezza >=1;
- **isspace**() restituisce True se la stringa contiene solo spazi bianchi (spazio, \n, \t) e ha lunghezza >=1.

Un esempio di applicazione dei metodi per il test delle stringhe è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
s = "abracadabra"
if s.islower():
    print(s, "contiene tutti caratteri minuscoli")
s = "ABRACADABRA"
if s.isupper():
    print(s, "contiene tutti caratteri maiuscoli")
s = "3sere"
if s.isalnum():
    print(s, "contiene sia caratteri alfabetici che numerici")
if s.isalpha():
    print(s, "contiene solo caratteri alfabetici")
```

Il risultato dell'elaborazione precedente è il seguente:

```
abracadabra contiene tutti caratteri minuscoli
ABRACADABRA contiene tutti caratteri maiuscoli
3sere contiene sia caratteri alfabetici che numerici
```

I metodi per la **modifica delle stringhe** più importanti sono:

- **lower**() restituisce una stringa con tutti i caratteri minuscoli;
- **upper**() restituisce una stringa con tutti i caratteri maiuscoli;
- **lstrip**() restituisce una stringa con tutti gli spazi bianchi (spazio, \n, \t) iniziali rimossi;
- **rstrip**() restituisce una stringa con tutti gli spazi bianchi (spazio, \n, \t) finali rimossi;
- **strip**() restituisce una stringa con tutti gli spazi bianchi (spazio, \n, \t) iniziali e finali rimossi;
- **lstrip**(c), **rstrip**(c) e **strip**(c) restituiscono una stringa con tutti i caratteri indicati da c rimossi (iniziali, finali, iniziali e finali, rispettivamente).

I metodi lower() e upper() permettono di confrontare più facilmente due stringhe poiché si confrontano solo caratteri minuscoli o solo caratteri maiuscoli. Nel codice sorgente che segue, per la stessa stringa s che contiene:

- uno spazio a sinistra
- una x
- uno spazio
- i caratteri "ciao"
- uno spazio
- una x
- uno spazio

si richiamano i metodi appena presentati. Nella funzione print viene eliminato il separatore (stringa vuota) e vengono inseriti due barre verticali prima e dopo ciascuna stringa restituita dal metodo invocato per sottolineare l'effetto della chiamata al metodo stesso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
s = " x ciao x "
print("Stringa originale \t|",s,"|",sep="")
print("upper \t\t|",s.upper(),"|",sep="")
print("lstrip \t\t|",s.lstrip(),"|",sep="")
print("rstrip \t\t|",s.rstrip(),"|",sep="")
print("strip \t\t|",s.strip(),"|",sep="")
print("lstrip x  \t|",s.lstrip("x "),"|",sep="")
```

Il risultato dell'elaborazione precedente risulta essere:

```
Stringa originale       | x ciao x |
upper           | X CIAO X |
lstrip          |x ciao x |
rstrip          | x ciao x|
strip           |x ciao x|
lstrip x        |ciao x |
```

Spesso nei programmi è necessario eseguire **ricerche di sottostringhe**. Un esempio è quello della ricerca di una parola (sottostringa) in un testo scritto (documento) con un editor di testi.

I metodi offerti da Python per cercare e sostituire sottostringhe in stringhe più grandi sono:

- **endswith**(sottostringa) restituisce True se la stringa termina con la sottostringa passata come parametro;
- **startsswith**(sottostringa) restituisce True se la stringa inizia con la sottostringa passata come parametro;
- **find**(sottostringa) restituisce l'indice della prima occorrenza (indice del primo carattere) della sottostringa nella stringa; se la sottostringa non è presente nella stringa, la funzione find() restituisce -1;
- **replace**(vecchia,nuova) restituisce una copia della stringa in cui tutte le occorrenze della sottostringa *vecchia* sono sostituite dalla sottostringa *nuova*.

Il codice sorgente seguente mostra un esempio di uso del metodo **endswith**(). In particolare, inserendo il nome completo di un file, il programma esamina l'estensione del file e nel caso in cui l'estensione sia .txt o .py comunica, rispettivamente che è un file di testo o un file Python. In caso contrario, visualizza il messaggio "Tipo file sconosciuto".

```
nomefile = input("Inserire il nome del file: ")
if nomefile.endswith(".txt"):
    print("File di testo")
elif nomefile.endswith(".py"):
    print("File Python")
else:
    print("Tipo file sconosciuto")
```

Il risultato dell'elaborazione del codice precedente per un nome di file di esempio "prova.py" è il seguente:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Inserire il nome del file: prova.py
File Python
```

Il codice sorgente seguente mostra un esempio di uso del metodo **find**(). In particolare, si vuole ricercare la sottostringa "vita" all'interno di una stringa. Se il programma trova la sottostringa, comunica la prima occorrenza della sottostringa nella stringa più grande, altrimenti comunica di non aver trovato la parola.

```
stringa = """Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscura,
ché la diritta via era smarrita."""
subs = "vita"
i = stringa.find(subs)
if i==-1:
    print("Parola",subs,"non trovata")
else:
    print("Prima occorrenza della parola",subs,"trovata nell'indice",i)
```

Il risultato dell'elaborazione del programma precedente risulta essere il seguente:

```
Prima occorrenza della parola vita trovata nell'indice 31
```

Il codice sorgente seguente mostra un esempio di uso del metodo **replace**(). In particolare, il programma ha lo scopo di correggere la parola errata "cammino" con la parola corretta "cammin".

```
stringa = """Nel mezzo del cammino di nostra vita
mi ritrovai per una selva oscura,
ché la diritta via era smarrita."""
subs = "cammino"
print(stringa.replace(subs,"cammin"))
```

Il risultato dell'elaborazione del programma precedente risulta essere il seguente:

```
Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscura,
ché la diritta via era smarrita.
```

Python offre, inoltre, la possibilità per le stringhe, come per le sequenze, di utilizzare l'**operatore di ripetizione**.

Nel primo esempio di uso dell'operatore di ripetizione, si ripete la stringa "a" per 5 volte.

```
stringa = "a"*5
print(stringa)
```

La visualizzazione a schermo di stringa risulta essere:

```
aaaaa
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel secondo esempio di uso dell'operatore di ripetizione, si utilizza un ciclo for per ripetere un numero incrementale di volte la stringa "a" e un secondo ciclo per ripetere un numero decrementale di volte la stringa "a".

```
for i in range(1,5):
    print("a"*i)
for i in range(5,0,-1):
    print("a"*i)
```

Il risultato dell'elaborazione del programma precedente risulta essere la stampa di 9 stringhe con un diverso numero di a, prima crescente e poi decrescente.

```
a
aa
aaa
aaaa
aaaaa
aaaa
aaa
aa
a
```

Il metodo **split**() delle stringhe restituisce una lista contenente le parole della stringa (di default separate da spazi nella stringa originale). È possibile prevedere anche altri caratteri di spaziatura indicandoli come parametri del metodo. Nell'esempio seguente si riportano due casi di uso del metodo split:

1. il primo con lo spazio di default per una stringa;
2. il secondo con lo slash come separatore per una data.

```
s = """Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscura,
ché la diritta via era smarrita."""
print(s.split())

data = "01/08/2021"
print(data.split('/'))
```

Il risultato dell'elaborazione precedente è la visualizzazione di due liste di stringhe:

```
['Nel', 'mezzo', 'del', 'cammin', 'di', 'nostra', 'vita', 'mi', 'ritrovai', 'per
', 'una', 'selva', 'oscura,', 'ché', 'la', 'diritta', 'via', 'era', 'smarrita.']
['01', '08', '2021']
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il metodo **join**() unisce liste di stringhe a formare un unico oggetto stringa. Il contenuto originale della stringa su cui si chiama il metodo join fa da separatore tra gli elementi della lista. Nel codice sorgente che segue, il metodo join userà la stringa s uguale a ", " come separatore degli elementi della lista che verranno uniti in un'unica stringa.

```
lista = ["ciao","casa","mare","montagna"]
s = ", "
print(s.join(lista))
```

Il risultato dell'elaborazione precedente risulta essere la visualizzazione di un'unica stringa:

```
ciao, casa, mare, montagna
```

L'uso combinato delle funzioni split() e join() permette di eliminare da una stringa spazi indesiderati come nel caso mostrato di seguito.

```
stringa = "In questa   stringa\t   ci sono    spazi\n messi a  caso "
s = " "
print(s.join(stringa.split()))
```

Effettivamente, come è possibile osservare di seguito, il numero di spazi, tabulazioni e new line viene regolarizzato con un solo spazio tra ciascuna coppia di parole.

```
In questa stringa ci sono spazi messi a caso
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esercizi con le stringhe – calcolo del codice fiscale

Un esercizio di manipolazione delle stringhe molto noto è quello del calcolo del codice fiscale. Il codice fiscale viene calcolato noti per una persona:

1. cognome;
2. nome;
3. sesso;
4. data di nascita;
5. luogo di nascita.

Il codice fiscale è una stringa di 16 caratteri alfanumerici. Si analizzano ora nel dettaglio gli algoritmi di calcolo di ciascuna porzione del codice fiscale.

4.1 Analisi del cognome – 3 caratteri alfabetici

Si considerano le consonanti del cognome (o dei cognomi, se ve ne è più di uno) nel loro ordine. Se il numero delle consonanti è insufficiente, si prelevano anche le vocali (se sono sufficienti le consonanti si prelevano la prima, la seconda e la terza consonante), sempre nel loro ordine.

Le vocali vengono eventualmente riportate dopo le consonanti (es. Rosi → RSO).

Nel caso in cui un cognome abbia meno di tre lettere, la parte di codice viene completata aggiungendo la lettera X (es. Fo → FOX).

Per poter distinguere le vocali dalle consonanti, si dichiara la stringa vocali in modo tale da utilizzare l'operatore IN per verificare che un carattere sia una vocale e l'operatore NOT IN per verificare che un carattere sia una consonante.

```
vocali = "aeiouAEIOU"
```

La funzione **calc_cognome**() restituirà la porzione di codice fiscale corrispondente al cognome della persona. Nella funzione si dichiarano due liste: *cons* per le consonanti del cognome e *voc* per le vocali del cognome. Con un ciclo for si analizzeranno tutte le lettere del cognome e si separeranno le vocali dalle consonanti. Attraverso la funzione join, si creerà una stringa che concatena le consonanti e le vocali del cognome e due caratteri x (si suppone che il cognome sia composto almeno da una lettera). La porzione di codice fiscale da restituire sarà uguale alla sottostringa (tutta in maiuscolo) dei primi 3 caratteri della stringa ottenuta tramite concatenazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def calc_cognome(cognome):
    cons = []
    voc = []
    for x in cognome:
        if x in vocali:
            voc.append(x)
        else:
            cons.append(x)
    codCognome = "".join(cons + voc +['x']*2)[0:3]
    return codCognome.upper()
```

4.2 Analisi del nome – 3 caratteri alfabetici

Si considerano le consonanti del nome (o dei nomi, se ve ne è più di uno) nel loro ordine. Se il nome contiene quattro o più consonanti, si scelgono la prima, la terza e la quarta consonante (per esempio: Gianfranco → GFR). Nel caso in cui il nome contenga tre consonanti, si considerano le prime tre in ordine (per esempio: Tiziana → TZN).

Se il nome non ha consonanti a sufficienza, si prendono anche le vocali. In ogni caso le vocali vengono riportate dopo le consonanti (per esempio: Luca → LCU).

Nel caso in cui il nome abbia meno di tre lettere, la parte di codice viene completata aggiungendo la lettera X.

La funzione **calc_nome**() restituirà la porzione di codice fiscale corrispondente al nome della persona. Nella funzione si dichiarano due liste: *cons* per le consonanti del cognome e *voc* per le vocali del cognome. Con un ciclo for si analizzeranno tutte le lettere del nome e si separeranno le vocali dalle consonanti.

Se la lista cons contiene più di 3 elementi, si elimina la seconda consonante (come previsto dall'algoritmo).

Attraverso la funzione join, si creerà una stringa che concatena le consonanti e le vocali del nome e due caratteri x (si suppone che il nome sia composto almeno da una lettera). La porzione di codice fiscale da restituire sarà uguale alla sottostringa (tutta in maiuscolo) dei primi 3 caratteri della stringa ottenuta tramite concatenazione.

```
def calc_nome(nome):
    cons = []
    voc = []
    for x in nome:
        if x in vocali:
            voc.append(x)
        else:
            cons.append(x)
    if len(cons) > 3:
        cons[1:2] = []
    codNome = "".join(cons + voc +['x']*2)[0:3]
    return codNome.upper()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4.3 Analisi della data di nascita e sesso – 5 caratteri alfanumerici

Per quanto riguarda l'anno di nascita a cui sono destinate due dei 5 caratteri, si prendono le ultime due cifre dell'anno di nascita.

Per il mese di nascita, invece, a cui è destinato uno dei 5 caratteri, ad ogni mese dell'anno viene associata una lettera in base alla tabella seguente:

Lettera	Mese	Lettera	Mese	Lettera	Mese
A	Gennaio	E	Maggio	P	Settembre
B	Febbraio	H	Giugno	R	Ottobre
C	Marzo	L	Luglio	S	Novembre
D	Aprile	M	Agosto	T	Dicembre

Per il giorno di nascita, a cui sono destinati gli ultimi 2 dei 5 caratteri, si prendono le due cifre del giorno di nascita (se è compreso tra 1 e 9 si pone uno zero come prima cifra).

Per i soggetti di sesso femminile, a tale cifra va sommato il numero 40. In questo modo il campo contiene la doppia informazione giorno di nascita e sesso. Si avrà, pertanto, la seguente casistica:

- i maschi avranno il giorno con cifra da 01 a 31;
- le donne la cifra relativa al giorno sarà da 41 a 71.

Prima di procedere con il codice sorgente della funzione che calcola i 5 caratteri corrispondenti a sesso e data di nascita, facciamo delle premesse.

Innanzitutto, andrà dichiarato come variabile globale un dizionario in cui si riporta la corrispondenza tra mese e lettera:

```
mesi = { '01':'A','02':'B','03':'C','04':'D',
         '05':'E','06':'H','07':'L','08':'M',
         '09':'P','10':'R','11':'S','12':'T'}
```

La data di nascita che verrà utilizzata sarà uguale ad una lista corrispondente alla data scomposta, tramite la funzione split, in giorno, mese e anno.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
data = input("Inserire data di nascita (gg/mm/anno): ").split('/')
```

La funzione **adj_data**() che restituisce la porzione di codice fiscale corrispondente al sesso e alla data di nascita:

- preleva le ultime due cifre ([2:]) del terzo elemento ([2]) della lista data corrispondente all'anno;
- calcola, mediante corrispondenza nel dizionario mesi all'indice pari al secondo elemento ([1]) della lista data, la lettera corrispondente;
- preleva il giorno di nascita corrispondente al primo elemento ([0]) della lista data e, nel caso in cui il sesso non sia maschile, aggiunge 40 (si noti il doppio casting prima a intero per eseguire la somma e, successivamente, a stringa).

La porzione di codice fiscale che verrà restituita sarà uguale alla concatenazione delle stringhe anno, mese e giorno.

```
def adj_data(data, sesso):
    anno = data[2][2:]
    mese = mesi[data[1]]
    if sesso.upper() != "M":
        giorno = str(int(data[0]) + 40)
    else:
        giorno = data[0]
    return anno + mese + giorno
```

4.4 Analisi del Comune (o Stato) di nascita – 4 caratteri alfanumerici

Per i comuni si utilizza il codice catastale del comune di nascita. Per gli altri Stati si utilizza la lettera Z seguita dal numero identificativo dello Stato. In questo esercizio, per brevità, si analizzeranno solo i Comuni italiani.

Il formato del file di testo che contiene i codici catastali dei Comuni italiani prevede per ogni riga il nome del Comune e il corrispondente codice catastale separati da una virgola come nell'estratto che segue:

```
Agliè,A074
Airasca,A109
Ala di Stura,A117
Albiano d'Ivrea,A157
Almese,A218
Alpette,A221
Alpignano,A222
```

La funzione **calc_cod_comune**() restituisce il codice catastale del Comune passato come parametro. Si apre in lettura il file di testo "codici_comuni.csv". Per ogni riga del file di testo, viene calcolata la lista *comuni* ottenuta tramite il metodo split a cui viene passato come separatore la virgola.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Viene, dunque, "ripulita" e trasformata tutta in maiuscolo la stringa corrispondente al nome del Comune da ricercare. Tramite una struttura decisionale si verifica l'uguaglianza tra la stringa del Comune della riga attuale e quella de Comune da ricercare. Se il Comune viene trovato, si chiude il file e si restituisce il codice catastale corrispondente al Comune. Se, terminata l'analisi di tutto il file di testo, il Comune non fosse stato trovato, si restituisce la stringa "COMUNE ERRATO".

```
def calc_cod_comune(comune):
    codici = open("codici_comuni.csv", "r")
    for linea in codici:
        comuni = linea.split(',')
        if comuni[0].upper().strip() == comune.upper():
            codici.close()
            return comuni[1].upper().rstrip()
    codici.close()
    return "COMUNE ERRATO"
```

4.5 Carattere di controllo – 1 carattere alfabetico

A partire dai quindici caratteri alfanumerici ricavati in precedenza, si determina il carattere di controllo (indicato a volte come **CIN, Control Internal Number**) in base a un particolare algoritmo che opera in questo modo:

- si mettono da una parte i caratteri alfanumerici che si trovano in posizione dispari e da un'altra quelli che si trovano in posizione pari;
- i caratteri vengono, poi, convertiti in valori numerici secondo le seguenti tabelle di corrispondenza in base alla loro posizione (pari/dispari) nel codice fiscale:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

CARATTERI ALFANUMERICI DISPARI							
Carattere	Valore	Carattere	Valore	Carattere	Valore	Carattere	Valore
0	1	9	21	I	19	R	8
1	0	A	1	J	21	S	12
2	5	B	0	K	2	T	14
3	7	C	5	L	4	U	16
4	9	D	7	M	18	V	10
5	13	E	9	N	20	W	22
6	15	F	13	O	11	X	25
7	17	G	15	P	3	Y	24
8	19	H	17	Q	6	Z	23

CARATTERI ALFANUMERICI PARI							
Carattere	Valore	Carattere	Valore	Carattere	Valore	Carattere	Valore
0	0	9	9	I	8	R	17
1	1	A	0	J	9	S	18
2	2	B	1	K	10	T	19
3	3	C	2	L	11	U	20
4	4	D	3	M	12	V	21
5	5	E	4	N	13	W	22
6	6	F	5	O	14	X	23
7	7	G	6	P	15	Y	24
8	8	H	7	Q	16	Z	25

- i valori che si ottengono dai caratteri alfanumerici pari e dispari vanno sommati tra di loro e il risultato va diviso per 26;
- il resto della divisione fornirà il codice identificativo, ottenuto dalla seguente tabella di conversione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

RESTO							
Resto	**Lettera**	**Resto**	**Lettera**	**Resto**	**Lettera**	**Resto**	**Lettera**
0	A	**7**	H	**14**	O	**21**	V
1	B	**8**	I	**15**	P	**22**	W
2	C	**9**	J	**16**	Q	**23**	X
3	D	**10**	K	**17**	R	**24**	Y
4	E	**11**	L	**18**	S	**25**	Z
5	F	**12**	M	**19**	T		
6	G	**13**	N	**20**	U		

Prima di procedere con la stesura della funzione corrispondente, si definiscono tre dizionari corrispondenti alle tre tabelle precedenti:

1. tabella di corrispondenza carattere alfanumerico-valore per posizioni pari:

```
pari = { '0':1, '1':0, '2':5, '3':7, '4':9, '5':13,
         '6':15, '7':17, '8':19, '9':21, 'A':1, 'B':0,
         'C':5, 'D':7, 'E':9, 'F':13, 'G':15, 'H':17,
         'I':19, 'J':21, 'K':2, 'L':4, 'M':18, 'N':20,
         'O':11, 'P':3, 'Q':6, 'R':8, 'S':12, 'T':14,
         'U':16, 'V':10, 'W':22, 'X':25, 'Y':24, 'Z':23}
```

2. tabella di corrispondenza carattere alfanumerico-valore per posizioni dispari:

```
dispari = { '0':0, '1':1, '2':2, '3':3, '4':4, '5':5,
          '6':6, '7':7, '8':8, '9':9, 'A':0, 'B':1,
          'C':2, 'D':3, 'E':4, 'F':5, 'G':6, 'H':7,
          'I':8, 'J':9, 'K':10, 'L':11, 'M':12, 'N':13,
          'O':14, 'P':15, 'Q':16, 'R':17, 'S':18, 'T':19,
          'U':20, 'V':21, 'W':22, 'X':23, 'Y':24, 'Z':25}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. tabella di corrispondenza carattere alfanumerico-valore di controllo:

```
controllo = { 0:'A',1:'B',2:'C',3:'D',4:'E',5:'F',
              6:'G',7:'H',8:'I',9:'J',10:'K',11:'L',
              12:'M',13:'N',14:'O',15:'P',16:'Q',17:'R',
              18:'S',19:'T',20:'U',21:'V',22:'W',23:'X',
              24:'Y',25:'Z'}
```

La funzione **codice_controllo**() riceve come parametro i 15 caratteri del codice fiscale calcolati grazie alle funzioni precedentemente illustrate.

Vengono inizializzati a zero un indice, un contatore per le posizioni dispari e un contatore per le posizioni pari. Vengono inizializzate come liste vuote p e d.

Si analizzano, quindi, le posizioni di ciascun carattere nel codice fiscale e si aggiungono i caratteri in modo opportuno nella lista p (corrispondente alle posizioni pari) o nella lista d (corrispondente alle posizioni dispari).

Per ciascun carattere della lista delle posizioni pari, tramite il dizionario pari di corrispondenza, si calcolano e accumulano i valori numerici. Stessa cosa, viene effettuata per i caratteri della lista delle posizioni dispari.

Al termine del calcolo, si sommano i due accumulatori e si raccoglie il resto della divisione tra tale valore e 26. Tale resto verrà utilizzato come indice nel dizionario di controllo per estrarre la sedicesima e ultima lettera del codice fiscale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def codice_controllo(codFisc):
    indice = 0
    a = 0
    b = 0
    p = list()
    d = list()
    while indice < len(codFisc):
        if indice%2!=0:
            d.append(codFisc[indice])
        else:
            p.append(codFisc[indice])
        indice +=1
    for x in d:
        a +=  dispari[x.upper()]
    for y in p:
        b += pari[y.upper()]
    return controllo[((a+b)%26)].upper()
```

Nel calcolo del codice fiscale è possibile assistere al fenomeno dell'**omocodia**. Due diverse persone potrebbero avere uguali tutte e sedici le lettere/cifre generate usando questo schema (omocodia non trattata nell'esercizio). In questo caso, l'Agenzia delle Entrate provvede a sostituire sistematicamente i soli caratteri numerici (a partire dal carattere numerico più a destra) con una lettera, secondo la seguente tabella di corrispondenza e ricalcola codice di controllo:

Cifra	Lettera	Cifra	Lettera	Cifra	Lettera
0	L	4	Q	8	U
1	M	5	R	9	V
2	N	6	S		
3	P	7	T		

Per concludere il codice sorgente, scriviamo la funzione main() che servirà principalmente per raccogliere le informazioni della persona per il calcolo del codice fiscale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():

    cognomel = input("Inserire il cognome: ")
    cognome = cognomel.replace(" ","")
    nomel = input("Inserire il nome: ")
    nome = nomel.replace(" ","")
    sesso = input("Inserire sesso(F o M): ")
    while len(sesso)!=1:
        sesso = input("Inserire sesso(F o M): ")
    data = input("Inserire data di nascita (gg/mm/anno): ").split('/')
    comune = input("Inserire comune di nascita: ")

    cod_fisc = calc_cognome(cognome)+calc_nome(nome)+\
               adj_data(data, sesso)+calc_cod_comune(comune)
    print (cod_fisc+codice_controllo(cod_fisc))


main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).